

Yazılım tasarım ve mimarisi:

Yazılım nedir: Yazılım, sadece bilgisayar programlarından ibaret değildir. En genel tanımıyla, bir sistemin donanım bileşenleri dışında kalan her şeydir. Beş ana bileşenlerde oluşur.

Mantık(algoritma): Bilgisayarlaştırmak istenen işin akıl yürütme sürecidir.

Veri(bilgi): yazılımın üzerinde çalıştığı ham ve işlenmiş bilgilerdir.

Belge (dokümanlar): analiz, tasarım ve planlama gibi aşamalarda üretilen kayıtlı bilgilerdir.

İnsan: yazılımı geliştiren(takım) ve kullanan kişilerdir.

Program(kod): yazılımın ana çıktısı olan çalıştırılabilir komutlar bütünüdür.

Yazılım vs Donanım:

Üretim: Donanım fabrikada seri üretilirken, yazılım geliştirilir ve mühendislik disipliniyle oluşturulur.

Eskime: donanımın fiziksel bir ömrü vardır ve eskir. Yazılım fiziksel olarak eskimez; ancak yeni ihtiyaçları karşılayamazsa veya teknolojisi geride kalırsa “eski” kabul edilir.

Kopyalama: Donanımı kopyalamak maliyetliken, yazılımı kopyalamak çok daha kolay ve ucuzdur. Ancak, kritik sistemlerde hata payını azaltmak için yazılımın aynısını değil, aynı işi yapan farklı bir eş yazılımı yedeklemek gerekebilir.

Yazılım mühendisliği ve hedefleri:

Tanım: yazılım geliştirme, işletme ve bakım süreçlerine sistemli, düzenli ve ölçülebilir bir mühendislik yaklaşımının uygulanmasıdır.

Amaç: yazılım üretimindeki karmaşıklıkları gidermek ve kaliteli bir süreç yönetmektir.

Yazılım hataları ve maliyetleri:

Hataların yayılması: yazılımda yapılan bir hata, ne kadar geç fark edilirse düzeltme maliyeti o kadar artar. Örneğin analiz aşamasındaki bir hatanın maliyeti “1” birimse, işletim aşamasında bu maliyet “100” birime kadar çıkabilir.

Sinama(test): Bir program tüm olasılıklarla test etmek imkansızdır; bu yüzden sınırlı sayıda veri ile en iyi sonuç aranır.

Maliyet Dengesi: Günümüzde donanım fiyatları düşerken, yazılımın geliştirme ve bakım maliyetleri(insan gücü ve süreç karmaşıklığı nedeniyle) giderek artmaktadır.

Yazılım Sistemlerin Sınıflandırılması: Yazılımlar üç ana kritiklerle sınıflandırılır:

İşlevlerine göre: hesaplama, veri işleme, gömülü sistemler, kural temelli (Yapayzeka) yazılımlar.

Zamana göre: Toplu (çevrim dışı), çevrim-içi ve gerçek zamanlı sistemler.

Boyuta göre: küçük (pc oyunları), orta (cad yazılımları), büyük(işletim sistemleri) ve çok büyük (komuta kontrol sistemleri).

Yazılım kalite: kalite sağlama etkinlikleri; maliyetleri düşürmeyi, yönetimi kolaylaştırmayı ve standart sorunlarını gidermeyi hedefler.

Kalite kriterleri:

Güvenirlilik, kullanabilirlik, taşınabilirlik, verimlilik ve bakım kolaylığı gibi birçok faktörü içerir.

Yazılım yaşam döngüsü Nedir:

Yazılımın üretiminden kullanım dışı kalana kadar geçirdiği tüm aşamalardır. Önemli özellikleri şunlardır:

Döngüsel Yapı: gereksinimler sürekli değiştiği için süreç bir döngü olarak ele alınır.

Geri Dönüşler: Herhangi bir aşamada geriye dönmek ve tekrar ilerlemek mümkündür.

Doğrusal Değildir: sadece tek yönlü ve düz bir hat olarak düşünülmemelidir.

Yaşam döngüsünün temel adımları (çekirdek süreçler):

Planlama: personel/donanım gereksinimleri, fizibilite çalışması ve proje planı yapılır.

Çözümleme(analiz): gereksinimler ayrıntılı çıkarılır, mevcut sorunlar incelenir. UML diyagramları(use case, class vb.) bu aşamada başlar

Tasarım:

Belirlenen gereksinimlere göre sistem yapısı oluşturulur.

- **Mantıksal tasarım:** önerilen sistemin yapısı ve örgütsel değişiklikler.
- **Fiziksel Tasarım:** yazılım bileşenleri ve ayrıntıları.

Gerçekleştirim: kodlama, test etme ve kurulum aşamasıdır.

Bakım: hata giderme ve yeni eklentiler yapma sürecidir; yazılımın ömrü boyunca sürer.

Süreç Modelleri ve Çeşitleri:

Süreç modelleri, bu adımların hangi sırada ve nasıl uygulanacağını tanımlayan rehberlerdir.

Geleneksel Modeller:

Kodla ve düzelt(code and fix): planlama yoktur, direkt kodlanır. Bireysel işler için uygun ama büyük projelerde kod karmakarışık olur.

Gelişigüzel Model: 60' lı yıllarda kullanılan, kişiye bağımlı, yöntemsiz modeldir.

Barok Modeli: 70 lerde kullanılan doğrusal modeldir. Belgelemeyi ayrı bir süreç olarak sonunda yapar(günümüzde önerilmez).

Çağlayan (Waterfall) Modeli: Adımlar sırayla izlenir; biri bitmeden diğeri başlamaz. Gereksinimleri çok net ve kısa süreli projeler için uygundur.

V Modeli: sol taraf üretim, sağ taraf sinama (test) aşamalarıdır. Belirsizliğin az olduğu projelerde kullanıcı katılımını artırır.

Prototipleme: gereksinimleri netleştirmek için hızlıca kısmi bir sistem(Prototip) yapılır, müşteri memnun olana kadar düzeltilir. (yani ön bir örnek olarak düşünün bir işi yapmadan önce veya başladıktan sonra onu sunduğunuz ilk örnek.)

Yinelemeli ve artımlı modeller:

Helezonik (spiral)Model: risk analizi ön plandadır. Her döngü bir aşamayı temsil eder ve sürekli riskler değerlendirilir. Büyük projeler için iyidir ama pahalı ve karmaşıktır.

Evrimsel Geliştirme: başlangıç sistemi kurulur ve müşteri geri bildirimleriyle sürekli yeni versiyonlar geliştirilir. **Banka uygulamaları gibi geniş organizasyonlar için uygundur.**

Artırımsal geliştirme: yazılım, işlevleri artan parçalar (sürümler) halinde teslim edilir. “böl ve yönet” yaklaşımıdır.

Diğer modeller(sormayacam dedi ama yine de bilin):

Formal sistem geliştirme: Matematiksel temelli, hatasız kod yazmayı hedefleyen(z dili gibi) kritik sistemler içindir. Bileşen tabanlı Model: hazır bileşenleri birleştirerek sistem oluşturma esasına dayanır.

Metodolojiler: bir süreç modeli ile çeşitli belirtim yöntemlerini uyumlu bir bütündür.

Örnek: yourdon yapısal sistem tasarımı, çağlayan modelini temel alır ve yaygın kullanılır.

Bileşik süreç (Unified Process –UP): Nesne yönelimli sistemler geliştirmek için kullanılan, yinelemeli(iteratif) ve artımlı bir yazılım geliştirme metodolojisidir.

Temel özellikleri:

Yinelemeli ve Artımlı(Iterative-Incremental): Proje küçük parçalara bölünür. Her yineleme sonunda çalışan bir ürün parçası(artım) ortaya çıkar.

Mimari Merkezli: Tasarımın en başında sistemin sağlam bir iskeleti(mimarisi) kurulmasına odaklanılır.

Kullanım senaryosu(Use-Case) Güdümlü: Sistemin ne yapacağı, kullanıcıların ihtiyaç duyduğu işlevler üzerinden tanımlanır.

Birleşik Sürecin Dört Ana Aşaması: Birleşik süreç, zaman çizelgesi üzerinde şu dört evreden geçer:

Başlangıç: Projenin kapsamı belirlenir, iş hedefleri tanımlanır ve riskler analiz edilir.

Hazırlık: Mimari planlanır, detaylı gereksinimleri analiz edilir ve en kritik riskler çözülür.

Yapılandırma: sistemin geri kalanının tasarlandığı ve kodlandığı ana üretim aşamasıdır.

Geçiş: Yazılımın kullanıcıya teslim edildiği, beta testlerinin ve eğitimlerin yapıldığı aşamadır.

Neden Tercih edilir: Geleneksel “Çağlayan” modelinin aksine, bileşik süreçte riskler erkenden fark edilir. Her yinelemede test yapıldığı için hatalar projenin sonunda değil, sürecin içinde çözülür. Bu da daha kaliteli ve kullanıcı ihtiyacına tam cevap veren bir yazılım ortaya çıkmasını sağlar.

Model Adı:	Temel yaklaşım:	Avantajları:	Dezavantajları:	En uygun olduğu proje örnekleri:
Gelişigüzel:	Model kullanmadan doğrudan kodlama ve düzeltme.	Az çaba gerektirir, kişisel kullanımda hızlıdır.	Karmaşıklık arttıkça yönetilemez, bakım zordur.	Tek kişilik, çok küçük ölçekli projeler.
Barok modeli:	Yaşam döngüsü adımlarının doğrusal ilerlemesi.	Belgeleme vurgusu vardır.	Geri dönüşler tanımlı değildir, günümüzde önerilemez.	1970'li yılların eski sistemleri.
Çağlayan(Waterfall):	Her aşama biter, onaylanır ve bir sonrakine geçilir.	Yönetimi ve takibi kolaydır, disiplin sağlar	Gereksinim değişikliğine kapalıdır; hatalar en sonda fark edilir.	Gereksinimleri çok net, kısa süreli projeler(örneğin derleyici yazımı.)
V Modeli:	Üretim ve sınaama(test) aşamaları karşılıklı ilerler.	Doğrulama ve geçerleme planları erkenden yapılır.	Risk analizi içermez, aşamalar arası tekrarla kapalıdır.	Belirsizliğin az olduğu BT ve kamu ihaleli projeleri.
Prototipleme:	Hızlıca geçici bir model oluşturulup müşteri onayı alınır.	Yanlış anlamaları önler, gereksinimleri netleştirir.	"Hızlı ve kirli" kodlama riski, belgeleme eksikliği.	Kullanıcı arayüzü(UI) yoğunluklu uygulamalar.
Helezonik(Spiral):	Risk odaklı, her döngüde prototip ve risk analizi yapılır.	En riskli kısımlar önce çözülür, hataları erken giderir.	Çok karmaşıktır, küçük projeler için pahalıdır.	Büyük ölçekli ve yüksek riskli kritik projeler.
Evrimsel Geliştirme:	Başlangıç sürümü hızla çıkarılır, müşteriyle geliştirilir.	Değişen gereksinimlere uyum sağlar, riskleri azaltır.	Süreç görünürlüğü azdır, sistem yapısı bozulabilir.	Çok birimli banka uygulamaları.
Artırımsal:	Yazılım, işlevleri artan sürümler halinde teslim edilir.	Önemli özellikler erken sunulur, proje başarısızlık riski düşer	Tüm sistemi baştan tanımlamak zordur, uzman personel ister.	Dönemlik ödevler veya parça parça teslim edilecek ticari yazılımlar.
Birleşik Süreç:	Nesne yönelimli, yinelemeli ve artırımsal modern bir yaklaşım.	Kalite ve risk yönetimi dahil tümleşik bir yapı sunar.	Uygulaması kapsamlıdır, iyi yapılandırmış yönetim gerektirir.	Modern, büyük ölçekli kurumsal nesne yönelimli projeler.

Önemli farklar ve notlar:

Belgeleme: Çağlayan modeli belgelendirmeyi sürecin doğal bir parçası görürken, Barok modeli ayrı bir aşama olarak ele alır.

Risk yönetimi: Risk analizinin en güçlü olduğu model helezonik (spiral) modeldir.

Müşteri Etkileşimi: Prototipleme ve Çevik yaklaşımlarda müşteri sürece en üst düzeyde dahil edilir.

Birleşik süreç (Unified Process): Bu model aslında Helezonik ve artırimsal modellerin modern bir metodoloji haline getirilmiş halidir; içerisinde ayrıntılı süreç tanımları ve yönetim modelleri barındırır.